

Regisztráció és bejelentkezés

Regisztráció

Regisztracio.jsx

A frontend fejlesztése során elkészítettem a rendszer regisztrációs oldalát React keretrendszer használatával. Az oldal lehetővé teszi új felhasználók számára a rendszerbe történő regisztrációt név, e-mail cím, jelszó, telefonszám és szerepkör megadásával. A fejlesztés során React useState hook segítségével kezeltem az űrlapmezők adatait, majd a megadott adatokat fetch API használatával továbbítottam a backend regisztrációs végpontja felé. A rendszer sikeres regisztráció esetén visszajelző üzenetet jelenít meg, majd automatikusan átirányítja a felhasználót a bejelentkezési oldalra. A felhasználói felület kialakításához külön hitelesites.css stíluslapot alkalmaztam, amely biztosítja az egységes és felhasználóbarát megjelenést. Az elkészült funkció működését a backend API-val összekapcsolva teszteltem, és sikeresen ellenőriztem az új felhasználók létrehozását.

```
import { Link, useNavigate } from "react-router-dom";
import { useState } from "react";
import "../styles/hitelesites.css";

function Regisztracio() {

  const [nev, setNev] = useState("");
  const [email, setEmail] = useState("");
  const [jelszo, setJelszo] = useState("");
  const [telefonszam, setTelefonszam] = useState("");
  const [szerepkor, setSzerepkor] = useState("felhasznalo");

  const navigate = useNavigate();

  const regisztracio = async (e) => {

    e.preventDefault();

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/regisztracio",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            nev,
            email,
            jelszo,
            szerepkor,
            telefonszam
          })
        }
      );
```

```
    }  
  );  
  
  const adat = await valasz.json();  
  
  alert(adat.uzenet);  
  
  if (adat.siker) {  
  
    navigate("/bejelentkezes");  
  
  }  
  
} catch (err) {  
  
  console.error(err);  
  
  alert("Hiba történt!");  
  
}  
  
};  
  
return (  
  
  <div className="auth-container">  
  
    <div className="auth-card">  
  
      <h2>Regisztráció</h2>  
  
      <form onSubmit={regisztracio}>  
  
        <input  
          type="text"  
          placeholder="Név"  
          value={nev}  
          onChange={(e) => setNev(e.target.value)}  
        />  
  
        <input  
          type="email"  
          placeholder="E-mail"  
          value={email}  
          onChange={(e) => setEmail(e.target.value)}  
        />  
  
        <input
```

Szakmai Gyakorlat II. – Frontend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
        type="password"
        placeholder="Jelszó"
        value={jelszo}
        onChange={(e) => setJelszo(e.target.value)}
      />

      <input
        type="text"
        placeholder="Telefonszám"
        value={telefonszam}
        onChange={(e) => setTelefonszam(e.target.value)}
      />

      <select
        value={szerepkor}
        onChange={(e) => setSzerepkor(e.target.value)}
      >
        <option value="felhasznalo">
          Felhasználó
        </option>

        <option value="admin">
          Admin
        </option>
      </select>

      <button type="submit">
        Regisztráció
      </button>

    </form>

    <p>

      Már rendelkezik fiókkal?{" "}

      <Link to="/bejelentkezes">
        Jelentkezzen be!
      </Link>

    </p>

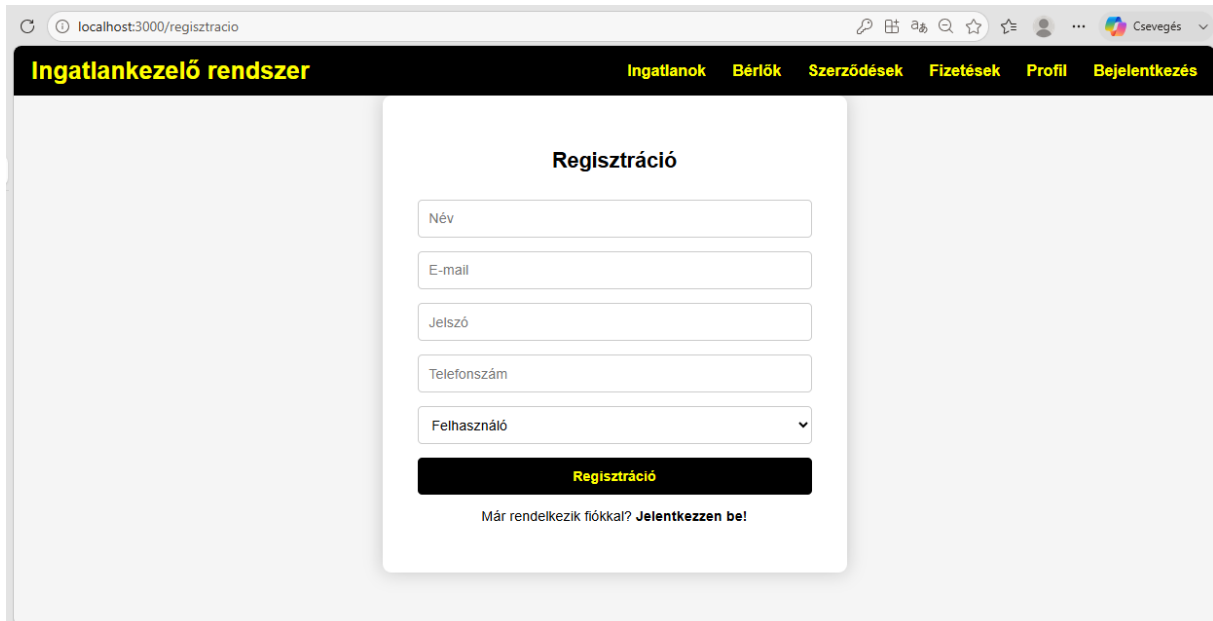
  </div>

</div>

);
```

```
}  
  
export default Regisztracio;
```

1. Részlet a regisztracio.jsx fájlból.



2. Kép a regisztráció oldalról.

Bejelentkezés.jsx

A frontend fejlesztése során elkészítettem a rendszer bejelentkezési felületét React keretrendszer használatával. Az oldal lehetővé teszi a felhasználók számára az e-mail címük és jelszavuk megadásával történő hitelesítést. A fejlesztés során React useState hook segítségével kezeltem a felhasználó által megadott adatokat, majd a fetch API alkalmazásával továbbítottam azokat a backend bejelentkezési végpontja felé. Sikeres bejelentkezés esetén a rendszer eltárolja a felhasználó adatait a böngésző localStorage tárhelyén, majd automatikusan átirányítja az Ingatlanok oldalra. Hibás bejelentkezési adatok esetén a felhasználó megfelelő hibaüzenetet kap a rendszerből. Az elkészült felület megjelenését a hitelesites.css stíluslap biztosítja, amely egységes és felhasználóbarát kialakítást nyújt a hitelesítési funkciók számára.

```
import { Link, useNavigate } from "react-router-dom";  
import { useState } from "react";  
import "../styles/hitelesites.css";  
  
function Bejelentkezés() {  
  
  const [email, setEmail] = useState("");  
  const [jelszo, setJelszo] = useState("");  
  
  const navigate = useNavigate();  
  
  const bejelentkezés = async (e) => {  
  
    e.preventDefault();
```

```
try {

    const valasz = await fetch(
        "http://localhost:3001/api/bejelentkezes",
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({
                email,
                jelszo
            })
        }
    );

    const adat = await valasz.json();

    if (adat.siker) {

        localStorage.setItem(
            "user",
            JSON.stringify(adat.user)
        );

        alert(adat.uzenet);

        navigate("/ingatlanok");

    } else {

        alert(adat.uzenet);

    }

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

return (
```

```
<div className="auth-container">

  <div className="auth-card">

    <h2>Bejelentkezés</h2>

    <form onSubmit={bejelentkezés}>

      <input
        type="email"
        placeholder="E-mail"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <input
        type="password"
        placeholder="Jelszó"
        value={jelszo}
        onChange={(e) => setJelszo(e.target.value)}
      />

      <button type="submit">
        Bejelentkezés
      </button>

    </form>

    <p>

      Nincs fiókja?{" "}

      <Link to="/regisztracio">
        Regisztráljon itt!
      </Link>

    </p>

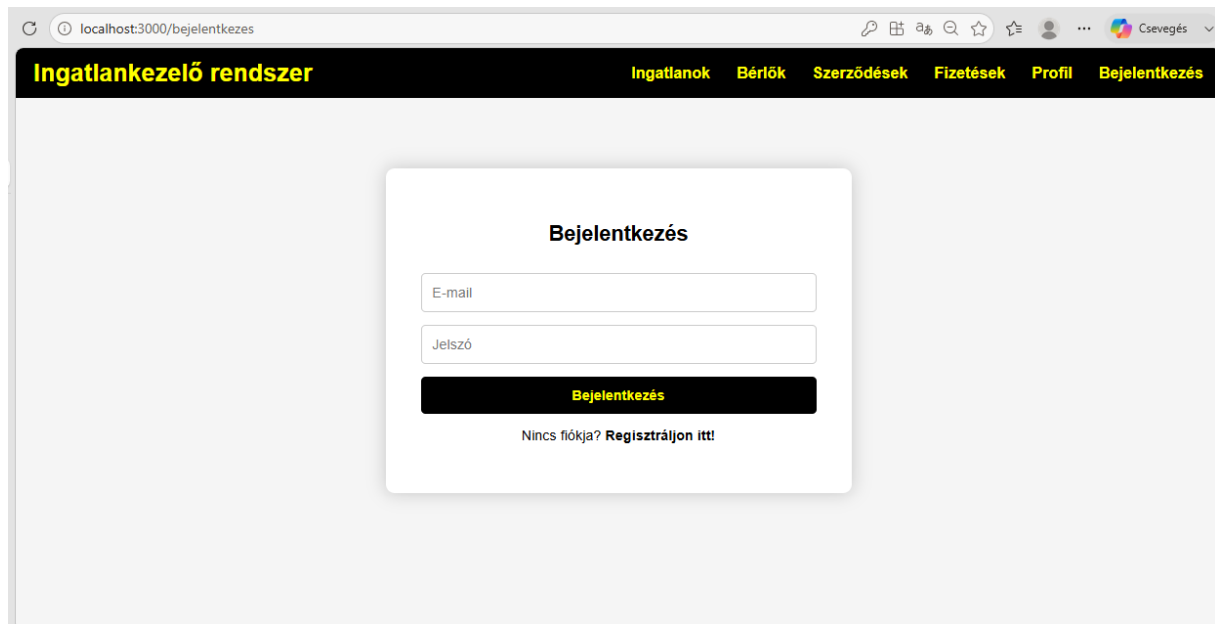
  </div>

</div>

);

}

export default Bejelentkezés;
```



4.Kép a bejelentkezés oldalról.

hitelesites.css

A frontend fejlesztése során elkészítettem a hitelesítési felületek megjelenését biztosító hitelesites.css stíluslapot. A stíluslap feladata a bejelentkezési és regisztrációs oldalak egységes, modern és felhasználóbarát megjelenésének kialakítása. A fejlesztés során középre igazítottam az űrlapokat, valamint egy kártya alapú elrendezést alkalmaztam, amely kiemeli a felhasználói adatbeviteli felületet. Meghatároztam a beviteli mezők, legördülő listák és gombok megjelenését, biztosítva azok könnyű használhatóságát és áttekinthetőségét. A rendszer színvilágához illeszkedve a gombok fekete háttérrel és sárga szöveggel jelennek meg, valamint egérmutató ráhelyezésekor vizuális visszajelzést adnak.

```
.auth-container {  
  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  
    min-height: 80vh;  
}  
  
.auth-card {  
  
    background-color: white;  
  
    width: 450px;  
  
    padding: 40px;  
  
    border-radius: 10px;  
  
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.2);  
}
```

```
}

.auth-card h2 {

    text-align: center;
    margin-bottom: 30px;

}

.auth-card input,
.auth-card select {

    width: 100%;

    padding: 12px;

    margin-bottom: 15px;

    border: 1px solid #ccc;
    border-radius: 5px;

    font-size: 16px;

    box-sizing: border-box;

}

.auth-card button {

    width: 100%;

    background-color: black;
    color: yellow;

    border: none;

    padding: 12px;

    border-radius: 5px;

    font-size: 16px;
    font-weight: bold;

    cursor: pointer;

}
```



```
.auth-card button:hover {  
    background-color: #222;  
}  
  
.auth-card p {  
    text-align: center;  
}  
  
.auth-card a {  
    color: black;  
    font-weight: bold;  
    text-decoration: none;  
}  
  
.auth-card a:hover {  
    text-decoration: underline;  
}
```

5. Részlet a hitelesites.css fájlból.